

Finanzas Personales

Documentacion tecnica

Documentacion generada para administracion, usuarios y despliegue.

Identidad: dashboard financiero moderno, permisos por modulo, correo verificado, PostgreSQL y .NET 8.

Documentacion tecnica - Finanzas Personales

Version: 1.0

Fecha: 2026-06-27

1. Resumen tecnico

La aplicacion es un sistema web financiero construido con ASP.NET Core MVC sobre .NET 8, PostgreSQL como base de datos, Dapper como micro ORM y Razor Views para la interfaz.

El sistema sigue una arquitectura MVC tradicional:

- Controllers: reciben solicitudes, validan permisos y ejecutan operaciones.
- Models: entidades y ViewModels principales.
- Views: pantallas Razor.
- Services: integraciones y logica transversal.
- Data: acceso a base de datos.

2. Tecnologias principales

- .NET 8.
- ASP.NET Core MVC.
- Razor Views.
- PostgreSQL.
- Dapper.
- Npgsql.
- BCrypt.Net-Next.
- ClosedXML.
- Bootstrap 5.
- Bootstrap Icons.
- Chart.js.
- DataProtection de ASP.NET Core.

3. Estructura del proyecto

Solucion:

- FinanzasPersonales.sln

Proyecto web:

- src/FinanzasPersonales.Web/FinanzasPersonales.Web.csproj

Carpetas principales:

- Controllers: controladores MVC.
- Views: vistas Razor.
- Models: entidades y modelos de pantalla.
- Services: servicios de correo, WhatsApp y asistente financiero.
- Data: clase de conexion a base de datos.
- wwwroot: archivos estaticos, estilos e iconos.
- sql: scripts de esquema.
- docs: documentacion fuente.
- output/pdf: documentos PDF generados.

4. Configuracion de aplicacion

Archivo:

- src/FinanzasPersonales.Web/appsettings.json

Actualmente no guarda secretos. La cadena de conexion se configura por variable de entorno:

```
$env:ConnectionStrings__Postgres="Host=localhost;Port=5432;Database=finanzas_personales;Username=postgres;Password=TU_PASSWORD"
```

Esto evita publicar contraseñas dentro del repositorio.

5. Inicio de la aplicacion

Archivo principal:

- Program.cs

Responsabilidades:

- Registrar servicios MVC.
- Registrar Db.
- Registrar servicios de correo y WhatsApp.
- Configurar autenticacion por cookies.
- Configurar DataProtection.
- Configurar rate limiting para acceso.
- Agregar cabeceras de seguridad.
- Configurar localizacion es-CO.
- Configurar rutas MVC.
- Ejecutar migraciones defensivas al iniciar.
- Crear usuario administrador inicial si no hay usuarios.

6. Seguridad implementada

Autenticacion

Se usa autenticacion por cookies de ASP.NET Core.

La cookie:

- Es HTTP only.
- Tiene expiracion.
- Usa SameSite Strict.
- En produccion exige cookie segura.

Autorizacion

La aplicacion usa claims para permisos:

- EsAdmin
- PermisoGastos
- PermisoPrestamos
- PermisoInversiones
- PermisoDirectivo
- PermisoAsistente
- PermisoCalendario

El BaseController valida que el usuario tenga permiso para acceder a cada modulo.

Antiforgery

Se agrega validacion antiforgery global para formularios MVC.

Rate limiting

Las rutas sensibles de autenticacion usan limitador fijo:

- Login.
- Recuperacion de contrasena.
- Restablecimiento.

Contrasenas

Las contrasenas se almacenan con BCrypt.

Politica:

- Minimo 12 caracteres.
- Mayuscula.
- Minuscula.
- Numero.

- Simbolo.

Verificacion de correo

Los tokens de verificacion se generan con criptografia segura, se guardan como hash SHA-256 y tienen vencimiento.

Proteccion de secretos

Las credenciales SMTP y tokens de WhatsApp se guardan cifrados con ASP.NET Core DataProtection.

Las llaves se almacenan en:

- App_Data/DataProtectionKeys

En despliegue se debe persistir esta carpeta o usar un almacen externo de llaves.

Cabeceras de seguridad

Se configuran:

- X-Content-Type-Options
- X-Frame-Options
- Referrer-Policy
- Permissions-Policy
- Content-Security-Policy

7. Base de datos

Motor:

- PostgreSQL.

Tablas principales:

- usuarios
- usuario_tokens
- usuario_pagos_suscripcion
- configuraciones_sistema
- configuraciones_usuario
- cuentas
- categorias
- movimientos
- gastos_periodicos
- presupuestos
- metas_ahorro
- aportes_meta
- personas
- prestamos

- prestamo_pagos
- tipos_inversion
- inversiones
- inversion_movimientos
- inversion_valoraciones

Nota: la tabla gastos_periodicos se mantiene por compatibilidad, pero funcionalmente representa movimientos recurrentes de tipo gasto o ingreso.

8. Indices importantes

La aplicacion crea indices para mejorar lectura:

- Usuarios por email.
- Usuarios por estado de suscripcion.
- Tokens vigentes por hash.
- Movimientos por usuario, tipo y fecha.
- Movimientos por categoria.
- Movimientos por cuenta.
- Cuentas por usuario, activo y tipo.
- Categorias por usuario, activo y tipo.
- Recurrentes por usuario, estado, tipo y fecha.
- Prestamos por usuario, estado y persona.
- Pagos de prestamos por usuario, prestamo y fecha.
- Metas por usuario y fecha.
- Inversiones por usuario, estado, tipo y fecha de retorno.

9. Modulos tecnicos

AccesoController

Gestiona:

- Login.
- Recuperacion de contrasena.
- Restablecimiento.
- Confirmacion de correo.
- Cambio de contrasena.
- Salida.

UsuariosController

Gestiona administracion de suscriptores:

- Crear usuario.
- Editar permisos.
- Registrar pagos.
- Suspender por mora.
- Reactivar.
- Reenviar verificación.

DashboardController

Gestiona tablero financiero de gastos:

- Totales.
- Presupuestos.
- Alertas.
- Recurrentes pendientes.
- Gráficas.
- Configuración de saldo anterior.

MovimientosController

Gestiona registros financieros:

- Ingresos.
- Gastos.
- Pagos de tarjeta.
- Transferencias.
- Filtros por fecha.

PeriodicosController

Gestiona movimientos recurrentes:

- Gasto recurrente.
- Ingreso recurrente.
- Registro real.
- Reprogramación automática.

PrestamosController

Gestiona préstamos:

- Tablero.
- Listado.
- Detalle.
- Pagos de interés.

- Abonos a capital.

PersonasController

Gestiona personas asociadas a prestamos y su resumen consolidado.

InversionesController

Gestiona portafolio:

- Inversiones.
- Detalle.
- Movimientos.
- Valoraciones.
- Tablero.

TiposInversionController

Gestiona tipos personalizables de inversion por usuario.

CalendarioController

Construye eventos financieros:

- Tarjetas.
- Recurrentes.
- Prestamos.
- Metas.
- Inversiones.

AsistenteController y AsistenteFinancieroService

Gestionan:

- Recordatorios.
- Informe mensual.
- Registro rapido.
- Mensajes para WhatsApp/correo.

ConfiguracionController

Gestiona:

- SMTP.
- WhatsApp Business Cloud API.
- Pruebas de envio.

10. Integraciones

SMTP

Servicio:

- EmailService

Se configura desde la interfaz de administrador. No requiere editar código.

Para Gmail se usa:

- smtp.gmail.com
- Puerto 587
- SSL/TLS activo.
- Contraseña de aplicación.

WhatsApp

Servicio:

- WhatsAppService

Usa WhatsApp Business Cloud API de Meta.

Requiere:

- Phone Number ID.
- Access Token.
- Version Graph API.
- Plantilla aprobada.
- Teléfono administrador.

11. Manejo visual

La interfaz usa:

- Bootstrap.
- Bootstrap Icons.
- CSS propio.
- Variables visuales para modo claro y oscuro.
- Tema púrpura/dorado.
- Diseño responsive.

La navegación se adapta por permisos.

12. Mantenimiento

Tareas recurrentes recomendadas:

- Revisar vulnerabilidades de paquetes.
- Verificar backups de PostgreSQL.

- Probar SMTP.
- Probar WhatsApp.
- Revisar logs.
- Confirmar que DataProtectionKeys se conservan.
- Revisar indices si crece el volumen.
- Ejecutar pruebas de login, permisos y recuperacion antes de publicar.

Comandos utiles:

```
dotnet build FinanzasPersonales.sln --no-restore
dotnet test FinanzasPersonales.sln --no-restore
dotnet list src/FinanzasPersonales.Web/FinanzasPersonales.Web.csproj package --vulnerable
--include-transitive
dotnet publish src/FinanzasPersonales.Web/FinanzasPersonales.Web.csproj -c Release -o
artifacts/publish
```

13. Riesgos tecnicos a controlar

- No publicar cadenas de conexion en appsettings.json.
- No perder llaves de DataProtection.
- No usar HTTP en produccion.
- No compartir usuario administrador.
- No dejar SMTP con contrasena normal de Gmail.
- No ejecutar en proveedores que no soporten ASP.NET Core directamente sin adaptacion.
- Configurar backups antes de recibir usuarios reales.